GA5-220501095-AA1-EV03

DOCUMENTACIÓN TÉCNICA INTEGRAL DEL PROYECTO VELANDIAX

ARQUITECTURA, BASE DE DATOS, API, FRONTEND, CHATBOT, GIT Y DESPLIEGUE

FICHA 3186657

APRENDIZ

NICOLÁS RUIZ GIRALDO

PRESENTADO AL INSTRUCTOR

JHON FAIBER CERQUERA SÁNCHEZ

ANÁLISIS Y DESARROLLO DE SOFTWARE

SERVICIO NACIONAL DE APRENDIZAJE

SENA REGIONAL HUILA

11/11/2026

Control del documento

Campo	Valor
Proyecto	VELANDIAX
Tipo	Memoria técnica integral y manual de continuidad
Presentación	Plantilla institucional SENA y adaptación técnica de APA, 7.ª edición
Aprendiz	Nicolás Ruiz Giraldo
Instructor	Jhon Faiber Cerquera Sánchez
Ficha	3186657
Repositorio	https://github.com/Nicolas-ruizg/DISTRIBUIDORA-VELANDIAX
Rama	main
HEAD local	a7104f1
Estado Git	## main...legacy/main [ahead 9]
Fecha de corte	11 de noviembre de 2026

Resumen

VELANDIAX es una plataforma comercial para una distribuidora de ropa que integra catálogo público, ventas por unidad y por paca, inventario por depósitos, clientes, pedidos, pagos, guías, seguimiento logístico, documentos imprimibles y atención por WhatsApp. El monorepositorio contiene una API REST en Python y FastAPI, una aplicación web en React y TypeScript, y un chatbot en Python conectado con WhatsApp Cloud API y Gemini. PostgreSQL conserva la información operativa mediante SQLAlchemy; las imágenes se cargan temporalmente a Supabase Storage. El despliegue usa Ubuntu, Nginx, systemd y HTTPS bajo velandiax.com.co. Este documento reconstruye la creación, funcionamiento y evolución del sistema a partir del código, los scripts SQL, el historial Git y el estado local pendiente de commit.

Palabras clave: comercio electrónico, inventario, FastAPI, React, PostgreSQL, SQLAlchemy, WhatsApp, trazabilidad, VELANDIAX.

Tabla de contenido

Control del documento 2

Resumen 2

Tabla de contenido 3

1. Introducción 6

1.1 Objetivo general 6

1.2 Objetivos específicos 6

1.3 Alcance 7

2. Contexto del negocio y capacidades 7

2.1 Capacidades funcionales 7

2.2 Reglas de negocio 8

3. Metodología y control de versiones 8

3.1 Prácticas observadas 8

4. Arquitectura general 9

4.1 Módulos 9

5. Lenguajes, frameworks y dependencias 10

5.1 Backend 10

5.2 Frontend 10

6. Estructura del repositorio 11

7. Construcción e instalación paso a paso 11

7.1 Prerrequisitos 11

7.2 Clonación 11

7.3 Base de datos 11

7.4 Backend 12

7.5 Frontend 12

7.6 Chatbot 12

8. Base de datos 12

8.1 Relaciones 12

8.2 Diccionario de datos completo . . . 13
8.3 Índices y restricciones . . . 15
8.4 Migraciones SQL . . . 16
9. Backend FastAPI 16
9.1 Autenticación y autorización . . . 16
9.2 Matriz de roles . . . 17
9.3 Inventario y concurrencia . . . 17
9.4 Pagos, guía y simulación . . . 17
9.5 Imágenes . . . 17
9.6 Catálogo completo de endpoints . . . 18
10. Frontend React 19
10.1 Tienda . . . 19
10.2 Panel . . . 20
10.3 Estado e impresión . . . 20
11. Chatbot de WhatsApp 20
11.1 Componentes . . . 20
11.2 Flujo . . . 21
12. Flujos de negocio 21
12.1 Alta de producto . . . 21
12.2 Pedido web . . . 21
12.3 Venta presencial . . . 22
12.4 Pago y despacho . . . 22
13. Despliegue productivo 22
13.1 DNS . . . 22
13.2 Instalación . . . 23
13.3 Actualización . . . 23
13.4 Diagnóstico . . . 23
14. Historial Git 23
14.1 Evolución funcional . . . 24

14.2 Cambios locales pendientes 24
15. Calidad, pruebas y validación 24
15.1 Cobertura recomendada 25
16. Seguridad y privacidad 25
16.1 Controles 25
16.2 Riesgos 26
17. Operación y mantenimiento 26
17.1 Respaldo 26
17.2 Lista de despliegue 26
18. Hallazgos y deuda técnica 27
19. Hoja de ruta 27
20. Conclusiones 27
Referencias 28
Apéndice A. Variables de entorno 29
Apéndice B. Estados 29
Apéndice C. Inventario de archivos rastreados 29

1. Introducción

El proyecto responde a la operación de una distribuidora de ropa con venta minorista y por paca. Su propósito es mantener una única fuente de información para productos, variantes, existencias, depósitos, usuarios, clientes, pedidos y envíos. Sobre esta base funcionan una tienda web, un panel administrativo y un chatbot de WhatsApp.

El análisis se realizó sobre el monorepositorio local, la rama main, los archivos rastreados, los cambios no confirmados, los scripts SQL y la configuración de despliegue. Se excluyen secretos reales. Cuando el código actual difiere del esquema consolidado o de los README, la diferencia se registra como hallazgo.

1.1 Objetivo general

Documentar de forma reproducible la arquitectura, construcción, reglas, operación y evolución de VELANDIAX para facilitar mantenimiento, auditoría, instalación y continuidad técnica.

1.2 Objetivos específicos

- Describir el negocio, los módulos y las reglas de productos, stock, ventas, pagos y envíos.
- Presentar tecnologías, dependencias, patrones y decisiones de diseño.
- Documentar tablas, campos, relaciones, restricciones, índices y migraciones.
- Catalogar endpoints, pantallas, roles e integraciones externas.
- Reconstruir Git por commits, ramas, remoto y cambios locales.
- Explicar instalación local, despliegue, operación, seguridad, pruebas y riesgos.

1.3 Alcance

La documentación cubre DISTRIBUIDORA-VELANDIAX, velandiax-admin, Velandiax-Chatbot y los archivos raíz. El repositorio registra 113 archivos. El inventario aproximado incluye 5.098 líneas TSX, 4.712 líneas Python, 570 líneas SQL y 489 líneas Markdown, sin contar dependencias o artefactos compilados.

2. Contexto del negocio y capacidades

VELANDIAX comercializa prendas en modalidad presencial y web. Cada producto puede tener variantes con talla, color, material u otros atributos libres; cada variante controla sus propios precios, estado, imágenes y stock. Los productos, usuarios y pedidos se relacionan con depósitos para segmentar la operación.

La tienda pública permite descubrir productos, ver ofertas, armar una paca, conservar favoritos, crear pedidos y consultar seguimiento. El backoffice administra catálogo, inventario, ventas, depósitos, usuarios, pagos, envíos, facturas y guías. El chatbot replica catálogo, carrito, checkout y seguimiento en WhatsApp.

2.1 Capacidades funcionales

- Catálogo público limitado a productos y variantes activas con stock.
- Código comercial manual, preparado para lectura por código de barras.
- Atributos variables almacenados en JSONB.
- Imágenes de producto y variante con principal, orden, texto alternativo y estado.
- Venta presencial y web con reglas distintas de pago y despacho.
- Trazabilidad por depósito, vendedor, responsable de inventario y cliente.
- Estados de pago CANCELADO, PAGO_50 y PAGO_TOTAL.
- Generación de guía al confirmar 50 % o pago total.
- Simulación logística cada 30 segundos mientras la API permanece activa.

- PDF de detalle, factura y guía para impresión térmica de 120 mm.
- Chatbot con Meta, menús, favoritos, compra por volumen y Gemini.
- Footer comercial, dos sedes, redes, TikTok y botón flotante de WhatsApp.

2.2 Reglas de negocio

Regla	Comportamiento actual
Conservación histórica	Productos y variantes se desactivan; no se eliminan físicamente.
Stock	No puede ser negativo; la venta bloquea variantes y descuenta en transacción.
Depósito	Un pedido pertenece a un depósito; el carrito no mezcla depósitos.
Alcance de usuario	No administradores operan sobre el depósito asignado.
Pago y guía	Solo PAGO_50 o PAGO_TOTAL habilitan guía.
Presencial	No requiere datos de envío ni regla de anticipo del 50 %.
Público	Se ocultan inactivos y variantes sin inventario.
Seguimiento	Consulta por celular o guía; excluye estados finales en la vista por celular.
Por paca	La interfaz usa Por paca; el campo técnico continúa como paquete_estatico.

3. Metodología y control de versiones

La evidencia corresponde a desarrollo iterativo e incremental orientado a funcionalidades. El sistema inició como solución full stack y recibió incrementos verticales de despliegue, datos, roles, depósitos, chatbot e impresión. No existe evidencia de sprints, ceremonias o artefactos suficientes para afirmar Scrum formal.

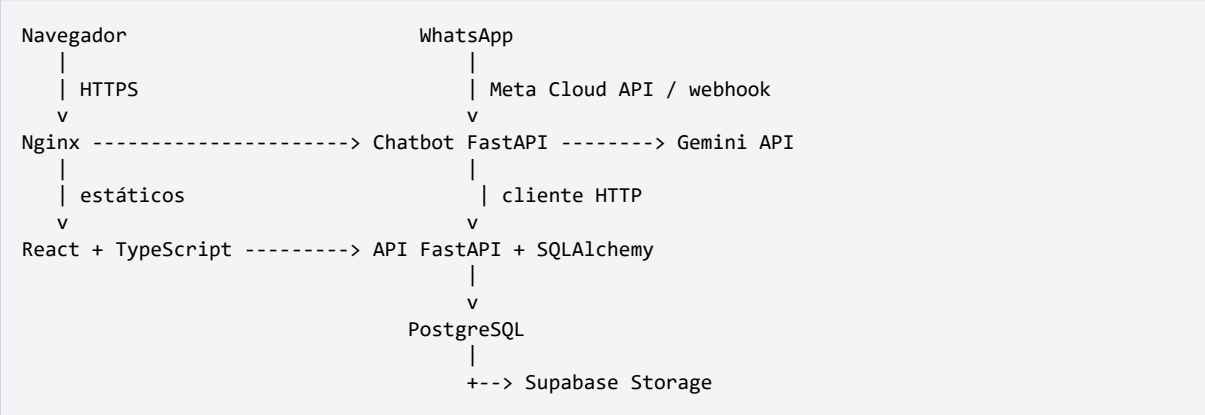
El repositorio usa una rama principal única, main, sin tags ni ramas feature registradas. Esta estrategia equivale a trunk-based simplificado y resulta viable para un equipo pequeño, siempre que se adopten commits frecuentes, revisión y protección de rama.

3.1 Prácticas observadas

- Monorepositorio para backend, frontend, chatbot y despliegue.
- Arquitectura por capas y módulos por capacidad.
- Contratos de datos con Pydantic y TypeScript.

- SQLAlchemy y transacciones para persistencia.
- Configuración por variables de entorno.
- RBAC aplicado en frontend y backend.
- Scripts SQL incrementales y esquema consolidado manual.
- Compilación de producción y pruebas versionadas del chatbot.

4. Arquitectura general



Nginx entrega el build de React y reenvía /api a Uvicorn en 127.0.0.1:8001. FastAPI concentra reglas y persistencia. El chatbot recibe eventos de Meta, consulta el catálogo real mediante la API y puede crear pedidos con una cuenta de ventas. Gemini interpreta lenguaje natural; SalesBot valida toda acción contra catálogo y estado conversacional.

4.1 Módulos

Módulo	Responsabilidad	Tecnologías
DISTRIBUIDORA-VELANDIA X	API, seguridad, negocio, persistencia, envíos y SQL.	Python, FastAPI, SQLAlchemy, PostgreSQL
velandiax-admin	Tienda pública y panel administrativo.	React, TypeScript, Vite, Tailwind
Velandiax-Chatbot	Webhook, canal conversacional, IA y backend.	Python, FastAPI, HTTPX, Meta, Gemini
DEPLOYMENT.md	Instalación y operación productiva.	Ubuntu, Nginx, systemd, Certbot
.vscode	Tareas para desarrollo local.	Visual Studio Code

5. Lenguajes, frameworks y dependencias

Capa	Tecnología	Versión declarada	Uso
Backend	Python	3.11+	API, negocio y automatización.
API	FastAPI	0.104.1	REST, DI, OpenAPI y validación.
ORM	SQLAlchemy	2.0.23	Modelos, sesiones y transacciones.
BD	PostgreSQL	Servidor compatible	Relaciones, índices y JSONB.
Esquemas	Pydantic	2.5.0	Entradas y salidas.
Seguridad	PyJWT / bcrypt	2.13.0 / 4.1.1	JWT y hash de claves.
Frontend	TypeScript	~6.0.2	Tipado del cliente.
UI	React	19.2.6	Componentes y contextos.
Build	Vite	8.0.12	Desarrollo y producción.
Estilos	Tailwind CSS	4.3.1	Diseño responsivo.
HTTP	Axios	1.17.0	Consumo e interceptores.
Documentos	jsPDF	4.2.1	Factura y guía.
Chatbot	HTTPX	0.28.1	Clientes HTTP asíncronos.
IA	Gemini API	Configurable	Interpretación de intención.
Servidor	Ubuntu/Nginx/systemd	Ubuntu 24	Hosting y supervisión.
Versiones	Git/GitHub	main	Historia y colaboración.

5.1 Backend

requirements.txt fija FastAPI, Uvicorn, psycopg2-binary, SQLAlchemy, PyJWT, bcrypt, python-dotenv, Pydantic, python-multipart, pytest y Pillow. El acceso es síncrono mediante Session de SQLAlchemy y psycopg2.

5.2 Frontend

React Router define navegación; Axios centraliza HTTP; Lucide React provee iconos; jsPDF genera documentos; Tailwind implementa el lenguaje visual blanco y negro. React Query está instalado, aunque la mayoría de cargas usa useEffect y estado local.

6. Estructura del repositorio

```
VELANDIAX/
|-- DISTRIBUIDORA-VELANDIAX/
|   |-- database/      modelos, conexión, DDL, migraciones y seed
|   |-- routes/        auth, catálogo, pedidos, usuarios y envíos
|   |-- schemas/       contratos Pydantic
|   |-- services/      simulador logístico
|   `-- utils/         JWT
|-- velandiax-admin/src/
|   |-- api, context, components y features
|-- Velandiax-Chatbot/
|   |-- app, data y tests
|-- DEPLOYMENT.md
`-- README.md
```

7. Construcción e instalación paso a paso

7.1 Prerrequisitos

- Git, Python 3.11+, Node.js 20/22, npm y PostgreSQL.
- Ubuntu 24, Nginx, systemd y Certbot para producción.
- Supabase Storage durante la etapa actual de imágenes.
- Aplicación Meta/WhatsApp Business y Google AI Studio para integraciones.

7.2 Clonación

```
git clone https://github.com/Nicolas-ruizg/DISTRIBUIDORA-VELANDIAX.git
cd DISTRIBUIDORA-VELANDIAX
git switch main
git status
```

7.3 Base de datos

1. Crear base y usuario PostgreSQL.
2. Ejecutar database/schema_final.sql.
3. Aplicar database/add_stock_variantes.sql, todavía pendiente de integrar al esquema final.
4. Renombrar DEP001 y DEP002.
5. Crear usuarios mediante API para almacenar bcrypt.
6. Usar seed_demo_ropa únicamente en demostración.

```
psql -U postgres -d velandiax -f database/schema_final.sql
psql -U postgres -d velandiax -f database/add_stock_variantes.sql
```

7.4 Backend

```
cd DISTRIBUIDORA-VELANDIAX
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
python -m uvicorn main:app --reload --host 127.0.0.1 --port 8001
```

7.5 Frontend

```
cd velandiax-admin
npm install
Copy-Item .env.example .env
npm run dev
```

VITE_API_URL usa http://127.0.0.1:8001 en local y /api en producción. Esto evita que el navegador del visitante intente consultar su propio localhost.

7.6 Chatbot

```
cd Velandiax-Chatbot
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
Copy-Item .env.example .env
uvicorn app.main:app --reload --port 8000
```

8. Base de datos

El diseño combina entidades relacionales para información estable con JSONB para atributos variables. NUMERIC se usa en dinero, salvo precio_anterior, que continúa como DOUBLE PRECISION. Los estados conservan historia de productos y variantes. Las llaves foráneas garantizan integridad entre catálogo, inventario, clientes, pedidos y envíos.

8.1 Relaciones

```
DEPOSITOS 1--N USUARIOS / PRODUCTO / PEDIDO
CATEGORIA 1--N PRODUCTO 1--N VARIANTES_PRODUCTO
PRODUCTO 1--N IMAGENES_PRODUCTO
VARIANTES_PRODUCTO 1--N IMAGENES_VARIANTE_PRODUCTO
CLIENTES 1--N PEDIDO 1--N ITEMS_PEDIDO N--1 VARIANTES_PRODUCTO
PEDIDO 1--0..1 ENVIOS
PRODUCTO N--N VARIANTES mediante PAQUETES_ESTATICOS_ITEMS
```

8.2 Diccionario de datos completo

Tabla	Campo	Tipo	Regla	Propósito
usuarios	id_usuario	INTEGER identity	PK	Identificador interno.
usuarios	id_deposito	VARCHAR(20)	FK nullable	Depósito asignado.
usuarios	nombre	VARCHAR(150)	NOT NULL	Nombre.
usuarios	email	VARCHAR(160)	UNIQUE	Credencial.
usuarios	codigo_cuenta	VARCHAR(20)	UNIQUE	VEND001/INV001.
usuarios	password_hash	VARCHAR(255)	NOT NULL	Hash bcrypt.
usuarios	rol	VARCHAR(30)	CHECK	ADMINISTRADOR/VENDEDOR/INVENTARIO.
usuarios	activo	BOOLEAN	DEFAULT true	Acceso.
usuarios	fecha_creacion	TIMESTAMP	DEFAULT now	Auditoría.
usuarios	ultimo_acceso	TIMESTAMP	nullable	Último login.
categoria	id_categoria	INTEGER identity	PK	Identificador.
categoria	nombre	VARCHAR(100)	NOT NULL	Nombre.
categoria	descripcion	TEXT	nullable	Descripción.
depositos	id_deposito	VARCHAR(20)	PK	Código manual.
depositos	nombre	VARCHAR(120)	nullable	Sede/bodega.
depositos	activo	BOOLEAN	DEFAULT true	Estado.
depositos	fecha_creacion	TIMESTAMP	DEFAULT now	Auditoría.
producto	id_producto	INTEGER identity	PK	ID técnico.
producto	id_categoria	INTEGER	FK	Categoría.
producto	id_deposito	VARCHAR(20)	FK	Depósito.
producto	codigo_producto	VARCHAR(60)	UNIQUE	Referencia manual.
producto	nombre	VARCHAR(200)	NOT NULL	Nombre.
producto	descripcion	TEXT	nullable	Detalle.
producto	paquete_estatico	BOOLEAN	DEFAULT false	Por paca.
producto	en_oferta	BOOLEAN	DEFAULT false	Oferta.
producto	precio_anterior	DOUBLE	nullable	Precio tachado.
producto	estado	VARCHAR(20)	CHECK	ACTIVO/INACTIVO.
variantes_producto	id_variante	INTEGER identity	PK	ID variante.
variantes_producto	id_producto	INTEGER	FK	Producto.
variantes_producto	precio_costo	NUMERIC(12,2)	NOT NULL	Costo.
variantes_producto	precio_minorista	NUMERIC(12,2)	NOT NULL	Precio venta.
variantes_producto	precio_mayorista	NUMERIC(12,2)	NOT NULL	Precio volumen heredado.
variantes_producto	stock	INTEGER	CHECK >= 0	Existencia.
variantes_producto	peso	NUMERIC(10,2)	nullable	Peso.

Tabla	Campo	Tipo	Regla	Propósito
variantes_producto	aplica_paquete	BOOLEAN	DEFAULT false	Elegible para paca.
variantes_producto	atributos	JSONB	nullable	Talla/color/material.
variantes_producto	estado	VARCHAR(20)	CHECK	ACTIVO/INACTIVO.
imagenes_producto	id_imagen	INTEGER identity	PK	Identificador.
imagenes_producto	id_producto	INTEGER	FK CASCADE	Producto.
imagenes_producto	url	VARCHAR(500)	NOT NULL	URL pública.
imagenes_producto	alt_text	VARCHAR(200)	nullable	Accesibilidad.
imagenes_producto	es_principal	BOOLEAN	DEFAULT false	Portada.
imagenes_producto	orden	INTEGER	CHECK >= 0	Orden.
imagenes_producto	estado	VARCHAR(20)	CHECK	Estado.
imagenes_producto	fecha_creacion	TIMESTAMP	DEFAULT now	Auditoría.
imagenes_variante_producto	id_imagen_variante	INTEGER identity	PK	Identificador.
imagenes_variante_producto	id_variante	INTEGER	FK CASCADE	Variante.
imagenes_variante_producto	url	VARCHAR(500)	NOT NULL	URL.
imagenes_variante_producto	alt_text	VARCHAR(200)	nullable	Accesibilidad.
imagenes_variante_producto	es_principal	BOOLEAN	DEFAULT false	Portada.
imagenes_variante_producto	orden	INTEGER	CHECK >= 0	Orden.
imagenes_variante_producto	estado	VARCHAR(20)	CHECK	Estado.
imagenes_variante_producto	fecha_creacion	TIMESTAMP	DEFAULT now	Auditoría.
clientes	id_cliente	INTEGER identity	PK	Identificador.
clientes	cedula	VARCHAR(30)	UNIQUE	Cédula/NIT.
clientes	celular	VARCHAR(20)	UNIQUE	Contacto/seguimiento.
clientes	nombre	VARCHAR(200)	NOT NULL	Nombre.
pedido	id_pedido	INTEGER identity	PK	Número interno.
pedido	id_cliente	INTEGER	FK	Cliente.
pedido	id_deposito	VARCHAR(20)	FK	Depósito.
pedido	id_usuario_venta	INTEGER	FK nullable	Vendedor.
pedido	id_usuario_inventario	INTEGER	FK nullable	Inventario.
pedido	canal_venta	VARCHAR(20)	CHECK	PRESENCIAL/WEB.
pedido	tipo_envio	VARCHAR(30)	CHECK	CONTRAENTREGA/GRATIS.
pedido	estado	VARCHAR(30)	NOT NULL	Estado operativo.
pedido	estado_pago	VARCHAR(20)	CHECK	CANCELADO/PAGO_50/PAGO_TOTAL.
pedido	total	NUMERIC(14,2)	DEFAULT 0	Total.
pedido	precio_envio	NUMERIC(12,2)	DEFAULT 0	Envío.
pedido	es_mayorista	BOOLEAN	DEFAULT false	Volumen heredado.
pedido	fecha	TIMESTAMP	DEFAULT now	Registro.

Tabla	Campo	Tipo	Regla	Propósito
items_pedido	id_item_pedido	INTEGER identity	PK	Detalle.
items_pedido	id_pedido	INTEGER	FK CASCADE	Pedido.
items_pedido	id_variante	INTEGER	FK	Variante.
items_pedido	cantidad	INTEGER	CHECK > 0	Unidades.
items_pedido	precio_unitario	NUMERIC(12,2)	NOT NULL	Precio congelado.
paquetes_estaticos_items	id_paquete	INTEGER	PK/FK	Producto paca.
paquetes_estaticos_items	id_variante	INTEGER	PK/FK	Variante incluida.
paquetes_estaticos_items	cantidad	INTEGER	CHECK > 0	Cantidad.
envios	id_envio	INTEGER identity	PK	ID logístico.
envios	id_pedido	INTEGER	FK UNIQUE	Pedido.
envios	transportadora	VARCHAR(120)	nullable	Operador.
envios	numero_guia	VARCHAR(120)	UNIQUE	Guía.
envios	estado	VARCHAR(30)	NOT NULL	Estado.
envios	nombre_destinatario	VARCHAR(200)	NOT NULL	Destinatario.
envios	celular_destinatario	VARCHAR(20)	nullable	Contacto.
envios	direccion	TEXT	NOT NULL	Dirección.
envios	ciudad	VARCHAR(100)	NOT NULL	Ciudad.
envios	departamento	VARCHAR(100)	nullable	Departamento.
envios	codigo_postal	VARCHAR(20)	nullable	Postal.
envios	notas	TEXT	nullable	Notas.
envios	costo	NUMERIC(12,2)	CHECK >= 0	Costo.
envios	fecha_despacho	TIMESTAMP	nullable	Salida.
envios	fecha_entrega	TIMESTAMP	nullable	Entrega.
envios	fecha_creacion	TIMESTAMP	DEFAULT now	Auditoría.

8.3 Índices y restricciones

Hay índices para categoría, código y depósito de producto; depósito de usuario; producto de variante; imágenes; cliente, depósito, vendedor y fecha del pedido; y detalles. UNIQUE cubre correo, código de cuenta, código de producto, cédula, celular y guía. CHECK valida roles, estados, cantidades, stock y costos.

8.4 Migraciones SQL

Archivo	Función
add_deposito_producto_tipo_envio.sql	Script incremental o esquema consolidado
add_deposito_usuario.sql	Script incremental o esquema consolidado
add_depositos_trazabilidad_ventas.sql	Script incremental o esquema consolidado
add_estado_pago_pedido.sql	Script incremental o esquema consolidado
add_estado_producto.sql	Script incremental o esquema consolidado
add_estado_variante_producto.sql	Script incremental o esquema consolidado
add_oferta_producto.sql	Script incremental o esquema consolidado
add_roles_y_codigo_producto.sql	Script incremental o esquema consolidado
add_stock_variantes.sql	Script incremental o esquema consolidado
create_envios.sql	Script incremental o esquema consolidado
create_imagenes_producto.sql	Script incremental o esquema consolidado
create_imagenes_variante_producto.sql	Script incremental o esquema consolidado
schema_final.sql	Script incremental o esquema consolidado

Hallazgo: add_stock_variantes.sql no está rastreado por Git y stock no aparece todavía en schema_final.sql, aunque el modelo y la aplicación dependen de él.

9. Backend FastAPI

main.py registra CORS, routers y un lifespan que inicia el simulador. connection.py usa URL.create, pool_pre_ping y sesiones cerradas por dependencia. Los modelos siguen SQLAlchemy 2.0 y Pydantic valida longitudes, rangos y enumeraciones.

9.1 Autenticación y autorización

POST /admin/login localiza el usuario activo, compara bcrypt, valida rol, actualiza ultimo_acceso y crea un JWT con usuario, depósito, código, rol, emisión y expiración. verificar_admin, verificar_inventario y verificar_ventas aplican permisos del lado servidor.

9.2 Matriz de roles

Función	Administrador	Vendedor	Inventario
Dashboard	Sí	No	No
Categorías	Completo	No	Crear/editar
Productos	Completo	Lectura API	Crear/editar en depósito
Registrar venta	Sí	Sí	No
Historial	Sí	No	No
Depósitos	Administrar	Asignado	Asignado
Usuarios	Administrar	No	No
Envíos	Administrar	No	No

9.3 Inventario y concurrencia

Al crear un pedido la API consulta variantes activas, bloquea sus filas con SELECT FOR UPDATE, valida depósito y existencia, descuenta stock y crea items en una transacción. El cambio local limita el bloqueo a VarianteProducto y usa selectinload para Producto, corrigiendo el error de PostgreSQL sobre FOR UPDATE en el lado anulable de un outer join.

9.4 Pagos, guía y simulación

CANCELADO significa pago no confirmado. PAGO_50 o PAGO_TOTAL permiten crear guía si existen datos de entrega. El simulador avanza PENDIENTE, PREPARANDO, DESPACHADO, EN_TRANSITO y ENTREGADO; registra fechas de despacho y entrega. Al ejecutarse dentro de la API puede duplicarse con varios workers.

9.5 Imágenes

Los endpoints multipart admiten JPG, PNG, WebP y GIF, controlan tamaño y suben a Supabase Storage. PostgreSQL almacena URL, alt, principal, orden y estado. La capa pública filtra imágenes inactivas.

9.6 Catálogo completo de endpoints

Método	Ruta	Acceso	Finalidad
GET	/	Público	Salud.
POST	/admin/login	Público	JWT.
GET	/admin/dashboard	Admin	Métricas y filtros.
GET	/categorias	Público	Lista.
GET	/categorias/{id}	Público	Detalle.
GET	/admin/categorias	Público actual	Alias.
POST	/admin/categorias	Admin/Inventario	Crear.
PUT	/admin/categorias/{id}	Admin/Inventario	Editar.
DELETE	/admin/categorias/{id}	Admin/Inventario	Eliminar.
GET	/productos	Público	Activos con stock.
GET	/productos/{id}	Público	Detalle.
GET	/admin/productos	Autenticado	Todos según depósito.
GET	/admin/productos/{id}	Autenticado	Detalle admin.
POST	/admin/productos	Admin/Inventario	Crear.
PUT	/admin/productos/{id}	Admin/Inventario	Editar/desactivar.
POST	/admin/productos/{id}/variantes	Admin/Inventario	Crear variante.
PUT	/admin/variantes/{id}	Admin/Inventario	Editar variante/stock.
POST	/admin/productos/{id}/imagenes	Admin/Inventario	URL producto.
POST	/admin/productos/{id}/imagenes/upload	Admin/Inventario	Subir producto.
PUT	/admin/productos/imagenes/{id}	Admin/Inventario	Editar imagen.
POST	/admin/variantes/{id}/imagenes	Admin/Inventario	URL variante.
POST	/admin/variantes/{id}/imagenes/upload	Admin/Inventario	Subir variante.
PUT	/admin/variantes/imagenes/{id}	Admin/Inventario	Editar imagen.
GET	/admin/pedidos	Admin	Historial.
POST	/pedidos	Público	Pedido web.
POST	/admin/pedidos	Admin/Vendedor	Venta manual.
GET	/admin/pedidos/{id}	Admin	Detalle.
PUT	/admin/pedidos/{id}	Admin	Actualizar/pago.
DELETE	/admin/pedidos/{id}	Admin	Eliminar.
POST	/admin/pedidos/{id}/envio	Admin	Crear envío.
GET	/admin/pedidos/{id}/envio	Admin	Consultar envío.
GET	/admin/envios	Admin	Listar.
GET	/envios/seguimiento/{guia}	Público	Guía.
GET	/pedidos/seguimiento	Público	Por celular.
GET	/admin/depositos	Autenticado	Listar según alcance.

Método	Ruta	Acceso	Finalidad
POST	/admin/depositos	Admin	Crear.
PUT	/admin/depositos/{id}	Admin	Editar.
DELETE	/admin/depositos/{id}	Admin	Desactivar.
GET	/admin/usuarios	Admin	Listar.
GET	/admin/usuarios/{id}	Admin	Detalle.
POST	/admin/usuarios	Admin	Crear.
PUT	/admin/usuarios/{id}	Admin	Editar.
DELETE	/admin/usuarios/{id}	Admin	Desactivar.

10. Frontend React

La SPA combina tienda y backoffice. StoreLayout envuelve la tienda con CartProvider y FavoritesProvider. AdminLayout organiza el panel; ProtectedRoute exige token y usuario; RoleRoute filtra por rol. Axios adjunta Bearer y limpia sesión ante 401. La paleta es blanco, negro, grises y verde de WhatsApp.

10.1 Tienda

Vista	Ruta	Función
HomeView	/	Banner, slider, recomendados, paca y TikTok.
ProductView	/producto/:id	Galería, atributos, stock, depósito y carrito.
CategoryView	/categoria/:id	Catálogo por categoría.
PackageBuilderView	/arma-tu-paca	Categorías y productos elegibles.
SaleView	/sale	Ofertas.
FavoritesView	/favoritos	Favoritos locales.
OrderTrackingView	/seguimiento	Pedidos activos por celular.
CartModal	Global	Carrito, checkout, pedido y WhatsApp.

10.2 Panel

Vista	Función
LoginView	Autenticación.
Dashboard	Métricas por tiempo, vendedor, referencia y depósito.
CategoriasView	Gestión de categorías.
ProductosView/Form	Código, depósito, oferta, variantes, stock e imágenes.
PedidoFormView	Venta presencial/web y búsqueda por código.
PedidosView/Detalle	Historial, pago, factura y guía.
DepositosView	Alta, edición y desactivación.
UsuariosView	Roles, códigos y depósito.

10.3 Estado e impresión

Carrito y favoritos se conservan en localStorage. El JWT usa admin_token y el usuario admin_user. jsPDF crea detalle, factura y guía térmica. La impresión depende del controlador y diálogo del navegador; para impresión silenciosa se requeriría QZ Tray o agente local.

11. Chatbot de WhatsApp

La segunda aplicación FastAPI verifica GET /webhook y recibe POST /webhook. Valida opcionalmente HMAC SHA-256, acepta rápido y procesa en BackgroundTasks. Una cola de 2.000 IDs deduplica mensajes dentro de una instancia.

11.1 Componentes

Archivo	Responsabilidad
main.py	Webhook, salud, deduplicación y coordinación.
bot.py	Máquina de estados, carrito y checkout.
catalog.py	Carga, búsqueda, categorías y recomendación.
backend.py	Catálogo, seguimiento, login y pedidos.
whatsapp.py	Texto, botones, listas, imágenes y lectura.
gemini.py	Intención estructurada.
storage.py	Pedidos JSONL con lock.
config.py	Entorno tipado.

11.2 Flujo

```
menú -> comprar / asesor / seguimiento
comprar -> categorías / ofertas / volumen
catálogo -> producto -> cantidad -> carrito
carrito -> editar / eliminar / checkout
checkout -> nombre -> documento -> ciudad -> dirección -> notas -> confirmar
confirmar -> backend -> PENDIENTE/CANCELADO -> cliente y asesor
```

Gemini solo interpreta intención y respuesta; SalesBot valida. Las sesiones están en memoria y los pedidos locales en JSONL, por lo que para escalar se recomienda Redis, PostgreSQL y un trabajador de colas.

12. Flujos de negocio

12.1 Alta de producto

1. Iniciar sesión como Admin o Inventario.
2. Seleccionar depósito y código manual.
3. Reutilizar datos si la referencia existe en historial.
4. Registrar categoría, oferta y Por paca.
5. Crear variantes con stock y atributos.
6. Subir imágenes.
7. Confirmar validaciones.

12.2 Pedido web

1. Agregar variantes con stock.
2. Impedir mezcla de depósitos.
3. Capturar nombre, documento, celular y dirección.
4. Enviar POST /pedidos.
5. Bloquear, validar y descontar stock.
6. Crear WEB/PENDIENTE/CANCELADO.
7. Abrir WhatsApp con número de pedido.

12.3 Venta presencial

1. Vendedor abre Registrar venta.
2. Selecciona PRESENCIAL.
3. Busca código o referencia.
4. Captura cliente.
5. Backend atribuye vendedor y depósito.
6. Transacción descuenta inventario.

12.4 Pago y despacho

1. Admin abre pedido.
2. Cambia a PAGO_50 o PAGO_TOTAL.
3. API crea guía si hay datos.
4. Simulador avanza estados.
5. Se imprime factura o guía.
6. Cliente consulta seguimiento.

13. Despliegue productivo

El entorno conocido usa Ubuntu 24 en 199.192.19.201. velandiax.com.co y www resuelven a esa IP. Los nameservers son dns1.registrar-servers.com y dns2.registrar-servers.com. Nginx sirve /var/www/VELANDIAX/velandiax-admin/dist; velandiax-api.service ejecuta Uvicorn en 127.0.0.1:8001 y /api actúa como proxy.

13.1 DNS

Tipo	Host	Destino
A	@	199.192.19.201
CNAME	www	velandiax.com.co

13.2 Instalación

```
sudo apt update
sudo apt install -y git nginx python3 python3-venv python3-pip certbot python3-certbot-nginx
cd /var/www
sudo git clone https://github.com/Nicolas-ruizg/DISTRIBUIDORA-VELANDIAX.git
cd DISTRIBUIDORA-VELANDIAX/DISTRIBUIDORA-VELANDIAX
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

13.3 Actualización

```
cd /var/www/VELANDIAX
git status
git pull --ff-only
cd DISTRIBUIDORA-VELANDIAX
source .venv/bin/activate
pip install -r requirements.txt
sudo systemctl restart velandiax-api.service
cd ../velandiax-admin
npm ci
npm run build
sudo nginx -t && sudo systemctl reload nginx
```

13.4 Diagnóstico

```
systemctl status velandiax-api.service --no-pager
journalctl -u velandiax-api.service -f
nginx -t
tail -f /var/log/nginx/error.log
curl https://velandiax.com.co/api/productos
```

14. Historial Git

El historial contiene 15 commits en main. No se encontraron tags ni ramas adicionales. El estado exacto de sincronización se incluye en Control del documento.

Fecha	Commit	Autor	Descripción
2026-05-27	764873e	Nicolas-ruizg	Initial commit
2026-05-28	63221ab	Nicolas-ruizg	Carga proyecto
2026-05-30	c0ca918	Nicolas-ruizg	avance dia 2
2026-05-30	ea758a4	Nicolas-ruizg	Feature: Initial commit with project structure and setup files
2026-06-11	44c51ad	Nicolas-ruizg	Feature: Update project structure and implement new features
2026-06-18	72032e9	Nicolas-ruizg	features
2026-06-24	fd62280	ArleyDev14	Initial VELANDIAX full stack project
2026-06-24	71f9925	ArleyDev14	Add production deployment guide
2026-06-24	46381c3	ArleyDev14	Update storefront home and seed demo data

Fecha	Commit	Autor	Descripción
2026-06-30	510db09	ArleyDev14	Add staff roles and manual product codes
2026-07-03	e740262	ArleyDev14	Add deposito access control and admin management
2026-07-08	b348c8f	ArleyDev14	Add Velandiax WhatsApp chatbot
2026-07-08	b6b845c	ArleyDev14	Update order PDF and paca labels
2026-11-11	c327603	Nicolas Ruiz Giraldo	Complete VELANDIAX platform and SENA documentation
2026-11-11	a7104f1	Nicolas Ruiz Giraldo	Merge existing DISTRIBUIDORA-VELANDIAX history

14.1 Evolución funcional

Commit	Aporte
fd62280	Base full stack: API, esquema, productos, pedidos, envíos, usuarios, tienda y panel.
71f9925	Guía de producción para Ubuntu, Nginx, systemd y HTTPS.
46381c3	Home, favicon, datos de demostración y Pillow.
510db09	Roles, códigos de cuenta, códigos manuales y protección por rol.
e740262	Depósitos, trazabilidad, canal, cédula, filtros, usuarios y depósitos.
b348c8f	Chatbot Meta/Gemini, catálogo, carrito, checkout y pruebas.
b6b845c	PDF, guía térmica y etiquetas Por paca.

14.2 Cambios locales pendientes

El directorio de trabajo contiene 0 archivos modificados y 0 rutas no rastreadas. Incluye inventario, endpoint público de pedido, corrección FOR UPDATE, TikTok, footer, WhatsApp, checkout, PDF y tipos. database/add_stock_variantes.sql y routes.zip todavía no están rastreados.

15. Calidad, pruebas y validación

Componente	Comando	Resultado observado
Backend	python -m compileall -q .	Correcto; advertencias de dependencias.
Frontend	npm run build	Correcto; 2.028 módulos.
Frontend	Bundle	JS principal 874,98 kB; advertencia >500 kB.
Chatbot	python -m pytest -q	No ejecutado: pytest no instalado en Python global y sin .venv.
Chatbot	tests/	Suite versionada para bot, webhook y WhatsApp.

15.1 Cobertura recomendada

- Unitarias para totales, volumen y estados.
- Integración PostgreSQL para concurrencia/stock.
- Autorización por rol y depósito.
- Contratos entre Pydantic y TypeScript.
- E2E de producto a entrega.
- CI con compileall, pytest, npm ci, lint y build.

16. Seguridad y privacidad

16.1 Controles

- bcrypt para contraseñas.
- JWT con expiración.
- RBAC en backend.
- CORS por entorno.
- Secretos en .env.
- Pydantic y restricciones SQL.
- Firma HMAC de Meta.
- Filtrado por depósito.

16.2 Riesgos

Prioridad	Riesgo	Recomendación
Alta	POST /pedidos público descuenta stock.	Rate limit, CAPTCHA, idempotencia y monitoreo.
Alta	Seguimiento por celular.	OTP, limitar campos y rate limit.
Alta	Cambios críticos sin commit.	Revisar, probar y confirmar.
Alta	Migraciones manuales/schema sin stock.	Alembic y reconciliación.
Media	JWT en localStorage.	CSP y evaluar cookie HttpOnly.
Media	Simulador en cada worker.	Trabajador único/cola.
Media	Chatbot en memoria.	Redis e idempotencia persistente.
Media	DELETE físico de pedidos/categorías.	Política de retención.
Baja	Bundle grande.	Lazy loading y code splitting.

17. Operación y mantenimiento

17.1 Respaldo

```
pg_dump -Fc -U velandiax_user -h 127.0.0.1 velandiax_db > velandiax_YYYYMMDD.dump
pg_restore --clean --if-exists -U velandiax_user -d velandiax_db velandiax_YYYYMMDD.dump
```

17.2 Lista de despliegue

- Git limpio y commit identificado.
- Respaldo restaurable.
- Migraciones aplicadas una vez.
- Dependencias reproducibles.
- Servicios activos.
- Build con VITE_API_URL=/api.
- nginx -t y TLS correctos.
- Pruebas de login, catálogo, pedido, stock, pago, guía y seguimiento.
- Monitoreo de logs.

18. Hallazgos y deuda técnica

- schema_final.sql no incluye stock.
- precio_mayorista y es_mayorista siguen presentes.
- paquete_estatico conserva nomenclatura anterior.
- No existe Alembic.
- GET /admin/categorias reutiliza lectura pública.
- Lecturas admin de producto permiten cualquier autenticado.
- El simulador se duplicaría con múltiples workers.
- El seed puede ser destructivo.
- No hay CI/CD, tags ni releases.
- README frontend continúa como plantilla Vite.
- Persisten textos históricos con codificación incorrecta.

19. Hoja de ruta

Horizonte	Acciones
Inmediato	Commit local; integrar stock; retirar routes.zip; pruebas chatbot; respaldo.
1-2 semanas	Alembic, rate limit, idempotencia, auditoría y CI.
1 mes	Redis, trabajador logístico, observabilidad y code splitting.
2-3 meses	Facturación formal, impresión administrada, códigos de barras y movimientos.
Continuo	Dependencias, OWASP, backups probados y documentación por release.

20. Conclusiones

VELANDIAX ya es una solución operativa y no solo una maqueta: integra dominio comercial, persistencia, inventario, roles, depósitos, canales web/WhatsApp, seguimiento e impresión. La arquitectura es comprensible y adecuada para el tamaño actual.

El riesgo principal es operativo: funcionalidades críticas permanecen sin commit y el esquema consolidado no representa completamente el modelo vigente. La prioridad es

reconciliar migraciones, confirmar cambios, endurecer endpoints públicos y automatizar validación y despliegue.

Referencias

FastAPI. (s. f.). FastAPI documentation. Recuperado el 11 de noviembre de 2026, de https://fastapi.tiangolo.com/

Git. (s. f.). Git reference. https://git-scm.com/docs

Google. (2026). Gemini API reference. https://ai.google.dev/api

Meta Platforms, Inc. (s. f.). WhatsApp Cloud API. https://developers.facebook.com/docs/whatsapp/cloud-api/

Open Worldwide Application Security Project. (2021). OWASP Top 10:2021. https://owasp.org/Top10/

PostgreSQL Global Development Group. (2026). JSON types. https://www.postgresql.org/docs/current/datatype-json.html

React Team. (2026). React reference overview. https://react.dev/reference/react

SQLAlchemy authors. (2026). SQLAlchemy 2.0 documentation. https://docs.sqlalchemy.org/en/20/

Vite contributors. (2026). Getting started. https://vite.dev/guide/

Ruiz Giraldo, N. (2026). DISTRIBUIDORA-VELANDIAX [Código fuente]. GitHub. https://github.com/Nicolas-ruizg/DISTRIBUIDORA-VELANDIAX

Apéndice A. Variables de entorno

Módulo	Variable	Propósito
Backend	DB_HOST, DB_PORT, DB_NAME	PostgreSQL.
Backend	DB_USER, DB_PASSWORD, DB_SSLMODE	Credenciales/TLS.
Backend	JWT_SECRET, JWT_ALGORITHM, JWT_EXPIRATION_HOURS	Tokens.
Backend	CORS_ORIGINS	Orígenes.
Backend	ENVIO_INTERVALO_SEGUNDOS	Simulador.
Backend	SUPABASE_URL, BUCKET, STORAGE_KEY	Imágenes.
Frontend	VITE_API_URL	Base API.
Chatbot	WHATSAPP_TOKEN, PHONE_NUMBER_ID	Meta.
Chatbot	VERIFY_TOKEN, META_APP_SECRET	Webhook.
Chatbot	BACKEND_API_URL/EMAIL/PASSWORD	Integración.
Chatbot	BACKEND_CREATE_ORDERS	Pedidos.
Chatbot	GEMINI_API_KEY, GEMINI_MODEL	IA.
Chatbot	STORE_URL, VOLUME_MIN_QUANTITY	Negocio.

Apéndice B. Estados

Entidad	Valores
Producto/variante/imagen	ACTIVO, INACTIVO
Rol	ADMINISTRADOR, VENDEDOR, INVENTARIO
Canal	PRESENCIAL, WEB
Pago	CANCELADO, PAGO_50, PAGO_TOTAL
Tipo envío	CONTRAENTREGA, GRATIS o null
Logística	PENDIENTE, PREPARANDO, DESPACHADO, EN_TRANSITO, ENTREGADO
Previstos	DEVUELTO, CANCELADO

Apéndice C. Inventario de archivos rastreados

Git rastrea 117 archivos. El listado permite verificar el alcance exacto de esta versión.

N.º	Ruta
1	.gitignore
2	.vscode/tasks.json
3	DEPLOYMENT.md
4	DISTRIBUIDORA-VELANDIAX/.env.example

N.º	Ruta
5	DISTRIBUIDORA-VELANDIAX/.gitignore
6	DISTRIBUIDORA-VELANDIAX/README.md
7	DISTRIBUIDORA-VELANDIAX/database/__init__.py
8	DISTRIBUIDORA-VELANDIAX/database/add_deposito_producto_tipo_envio.sql
9	DISTRIBUIDORA-VELANDIAX/database/add_deposito_usuario.sql
10	DISTRIBUIDORA-VELANDIAX/database/add_depositos_trazabilidad_ventas.sql
11	DISTRIBUIDORA-VELANDIAX/database/add_estado_pago_pedido.sql
12	DISTRIBUIDORA-VELANDIAX/database/add_estado_producto.sql
13	DISTRIBUIDORA-VELANDIAX/database/add_estado_variante_producto.sql
14	DISTRIBUIDORA-VELANDIAX/database/add_oferta_producto.sql
15	DISTRIBUIDORA-VELANDIAX/database/add_roles_y_codigo_producto.sql
16	DISTRIBUIDORA-VELANDIAX/database/add_stock_variantes.sql
17	DISTRIBUIDORA-VELANDIAX/database/connection.py
18	DISTRIBUIDORA-VELANDIAX/database/create_envios.sql
19	DISTRIBUIDORA-VELANDIAX/database/create_imagenes_producto.sql
20	DISTRIBUIDORA-VELANDIAX/database/create_imagenes_variante_producto.sql
21	DISTRIBUIDORA-VELANDIAX/database/models.py
22	DISTRIBUIDORA-VELANDIAX/database/schema_final.sql
23	DISTRIBUIDORA-VELANDIAX/database/seed_demo_ropa.py
24	DISTRIBUIDORA-VELANDIAX/main.py
25	DISTRIBUIDORA-VELANDIAX/requirements.txt
26	DISTRIBUIDORA-VELANDIAX/routes/__init__.py
27	DISTRIBUIDORA-VELANDIAX/routes/admin.py
28	DISTRIBUIDORA-VELANDIAX/routes/auth.py
29	DISTRIBUIDORA-VELANDIAX/routes/categorias.py
30	DISTRIBUIDORA-VELANDIAX/routes/depositos.py
31	DISTRIBUIDORA-VELANDIAX/routes/envios.py
32	DISTRIBUIDORA-VELANDIAX/routes/pedidos.py
33	DISTRIBUIDORA-VELANDIAX/routes/productos.py
34	DISTRIBUIDORA-VELANDIAX/routes/seguimiento.py
35	DISTRIBUIDORA-VELANDIAX/routes/usuarios.py
36	DISTRIBUIDORA-VELANDIAX/schemas/__init__.py
37	DISTRIBUIDORA-VELANDIAX/schemas/auth_schema.py
38	DISTRIBUIDORA-VELANDIAX/schemas/categoria.py
39	DISTRIBUIDORA-VELANDIAX/schemas/deposito.py
40	DISTRIBUIDORA-VELANDIAX/schemas/envio.py

N.º	Ruta
41	DISTRIBUIDORA-VELANDIAX/schemas/pedido.py
42	DISTRIBUIDORA-VELANDIAX/schemas/producto.py
43	DISTRIBUIDORA-VELANDIAX/schemas/usuario.py
44	DISTRIBUIDORA-VELANDIAX/services/__init__.py
45	DISTRIBUIDORA-VELANDIAX/services/simulador_envios.py
46	DISTRIBUIDORA-VELANDIAX/utils/__init__.py
47	DISTRIBUIDORA-VELANDIAX/utils/jwt_handler.py
48	README.md
49	Velandiax-Chatbot/.env.example
50	Velandiax-Chatbot/.gitignore
51	Velandiax-Chatbot/.vscode/tasks.json
52	Velandiax-Chatbot/README.md
53	Velandiax-Chatbot/app/__init__.py
54	Velandiax-Chatbot/app/backend.py
55	Velandiax-Chatbot/app/bot.py
56	Velandiax-Chatbot/app/catalog.py
57	Velandiax-Chatbot/app/config.py
58	Velandiax-Chatbot/app/gemini.py
59	Velandiax-Chatbot/app/main.py
60	Velandiax-Chatbot/app/storage.py
61	Velandiax-Chatbot/app/whatsapp.py
62	Velandiax-Chatbot/data/products.json
63	Velandiax-Chatbot/requirements-dev.txt
64	Velandiax-Chatbot/requirements.txt
65	Velandiax-Chatbot/tests/test_bot.py
66	Velandiax-Chatbot/tests/test_main.py
67	Velandiax-Chatbot/tests/test_whatsapp.py
68	documentacion/Documentacion_Tecnica_Integral_VELANDIAX_APA.pdf
69	documentacion/generar_pdf_apa.py
70	documentacion/sena_logo.png
71	velandiax-admin/.env.example
72	velandiax-admin/.gitignore
73	velandiax-admin/README.md
74	velandiax-admin/eslint.config.js
75	velandiax-admin/index.html
76	velandiax-admin/package-lock.json

N.º	Ruta
77	velandiax-admin/package.json
78	velandiax-admin/public/favicon.png
79	velandiax-admin/public/favicon.svg
80	velandiax-admin/public/icons.svg
81	velandiax-admin/src/App.tsx
82	velandiax-admin/src/api/axios.ts
83	velandiax-admin/src/assets/hero.png
84	velandiax-admin/src/assets/react.svg
85	velandiax-admin/src/assets/vite.svg
86	velandiax-admin/src/components/layout/AdminLayout.tsx
87	velandiax-admin/src/components/layout/ProtectedRoute.tsx
88	velandiax-admin/src/components/layout/RoleRoute.tsx
89	velandiax-admin/src/context/CartContext.tsx
90	velandiax-admin/src/context/FavoritesContext.tsx
91	velandiax-admin/src/features/admin/DepositosView.tsx
92	velandiax-admin/src/features/admin/UsuariosView.tsx
93	velandiax-admin/src/features/auth/LoginView.tsx
94	velandiax-admin/src/features/catalog/CategoriasView.tsx
95	velandiax-admin/src/features/catalog/ProductoFormView.tsx
96	velandiax-admin/src/features/catalog/ProductosView.tsx
97	velandiax-admin/src/features/dashboard/Dashboard.tsx
98	velandiax-admin/src/features/orders/PedidoDetalleView.tsx
99	velandiax-admin/src/features/orders/PedidoFormView.tsx
100	velandiax-admin/src/features/orders/PedidosView.tsx
101	velandiax-admin/src/features/storefront/CategoryView.tsx
102	velandiax-admin/src/features/storefront/FavoritesView.tsx
103	velandiax-admin/src/features/storefront/HomeView.tsx
104	velandiax-admin/src/features/storefront/OrderTrackingView.tsx
105	velandiax-admin/src/features/storefront/PackageBuilderView.tsx
106	velandiax-admin/src/features/storefront/ProductView.tsx
107	velandiax-admin/src/features/storefront/SaleView.tsx
108	velandiax-admin/src/features/storefront/StoreLayout.tsx
109	velandiax-admin/src/features/storefront/components/CartModal.tsx
110	velandiax-admin/src/index.css
111	velandiax-admin/src/main.tsx
112	velandiax-admin/src/types/index.ts

N.º	Ruta
113	velandiax-admin/src/utils/auth.ts
114	velandiax-admin/tsconfig.app.json
115	velandiax-admin/tsconfig.json
116	velandiax-admin/tsconfig.node.json
117	velandiax-admin/vite.config.ts